

--1. Crear un bloque PL que visualice el departamento de un empleado que se pida al usuario por teclado.

```
DECLARE
    v_empleado employees.employee_id%type := &cod;
    v_departamento departments.department_name%type;
BEGIN
    SELECT d.department_name INTO v_departamento
    FROM departments d
    INNER JOIN employees e on e.department_id = d.department_id
    WHERE e.employee_id = v_empleado;

    DBMS_OUTPUT.PUT_LINE('El departamento del empleado '||v_empleado||' es '||v_departamento);
END;
/
```

--2. Incrementar el salario 100€ a todos los trabajadores que sean 'IT_PROG', mediante un bloque anónimo PL, asignando dicho valor a una variable declarada.

```
DECLARE
    v_job_id employees.job_id%TYPE := 'IT_PROG';
BEGIN
    UPDATE employees
    SET salary = salary + 100
    WHERE job_id = v_job_id;
    DBMS_OUTPUT.PUT_LINE('Salarios actualizados para ' || SQL%ROWCOUNT || ' programadores.');
```

END;

/

--3. Crea un bloque de PL/SQL que inserte un nuevo registro en la tabla de empleados. Con las siguientes características:

- 3.a. Pedirá por teclado al usuario un código de empleado válido (employee_id)
- 3.b. Buscará al empleado que menos dinero gane que pertenezca al mismo departamento que el que nos han pasado y duplicará todos sus datos menos
 - 3.b.i. El id (sacarlo de la secuencia)
 - 3.b.ii. El nombre y apellidos que será: PEPITO GRILLO
 - 3.b.iii. La fecha de contratación será hoy.

-- Versión con dos consultas

```
DECLARE
    v_employee_id employees.employee_id%type;
    v_department_id departments.department_id%type;
    v_employee employees%ROWTYPE;
BEGIN
    --a. Pedir usuario ID empleado valido
    DBMS_OUTPUT.PUT_LINE('Introduce el codigo del empleado:');
    v_empleado := &ID_EMPLEADO;
    --b. Obtengo el dpto + el empleado menos gana
    SELECT department_id INTO v_department_id
    FROM employees
    WHERE employee_id = v_employee_id;

    SELECT * INTO v_employee
    FROM employees
    WHERE department_id = v_department_id AND salary =
    (
        SELECT MIN(salary)
        FROM employees
        WHERE department_id = v_department_id
    )
    FETCH FIRST 1 ROWS ONLY;

    --b. asignar datos nuevo empleado
```

```
INSERT INTO employees
VALUES (employees_seq.NEXTVAL, 'PEPITO', 'GRILLO', 'pepitogrillo@gmail.com',
v_employee.phone_number,
        SYSDATE , v_employee.job_id, v_employee.salary, v_employee.commission_pct,
        v_employee.manager_id, v_employee.department_id);
--COMMIT;
END;
/

-- Versión con una sola consulta
DECLARE
    v_input_id employees.employee_id%TYPE := &ID_EMPLEADO;
    v_employee employees%ROWTYPE;
BEGIN
    -- Obtener el registro del empleado que menos gana en ese departamento
    SELECT * INTO v_employee
    FROM employees e1
    WHERE e1.department_id = (SELECT department_id FROM employees WHERE employee_id = v_input_id)
        AND e1.salary = (SELECT MIN(e2.salary) FROM employees e2 WHERE e2.department_id =
e1.department_id)
    FETCH FIRST 1 ROWS ONLY;

    INSERT INTO employees
    VALUES (employees_seq.NEXTVAL, 'PEPITO', 'GRILLO', 'pepitogrillo@gmail.com',
v_employee.phone_number, SYSDATE, v_employee.job_id, v_employee.salary, v_employee.commission_pct,
v_employee.manager_id, v_employee.department_id);
END;
/

--4. Diseñar un bloque PL que introduciendo el código de un empleado por teclado, visualice el
suelo y su código, para posteriormente actualizar su comisión teniendo en cuenta que si su
salario es menor de 3.000 € su comisión será del 10% de este, si está entre 3.000 y 5.000 del 15%
y si es mayor de 5.000 el 20%. Posteriormente se visualizará su comisión actualizada.
DECLARE
    v_emp_id employees.employee_id%TYPE := &CODIGO_EMPLEADO;
    v_sal employees.salary%TYPE;
    v_comm employees.commission_pct%TYPE;
BEGIN
    SELECT salary INTO v_sal FROM employees WHERE employee_id = v_emp_id;

    DBMS_OUTPUT.PUT_LINE ('Suelo actual: ' || v_sal);

    UPDATE employees
    SET commission_pct = CASE
                        WHEN salary < 3000 THEN 0.1
                        WHEN salary BETWEEN 3000 AND 5000 THEN 0.15
                        ELSE 0.2
                    END
    WHERE employee_id = v_emp_id;
    --COMMIT
    SELECT commission_pct INTO v_comm FROM employees WHERE employee_id = v_emp_id;
    DBMS_OUTPUT.PUT_LINE ('Nueva comisión: ' || v_comm);
END;
/

--5. Introduciendo un año por teclado, decir si este es bisiesto o no.
DECLARE
    v_year NUMBER := &introduce_anio;
BEGIN
    IF (MOD(v_year, 4) = 0 AND MOD(v_year, 100) != 0) OR (MOD(v_year, 400) = 0) THEN
        DBMS_OUTPUT.PUT_LINE(v_year || ' es bisiesto.');
```

```
        DBMS_OUTPUT.PUT_LINE(v_year || ' NO es bisiesto.');
```

```
    END IF;
```

```
END;
```

```
/
```

--6. Diseñar un bloque de PL que le pida al usuario un código de empleado y que devuelva el mayor divisor del salario del empleado.

```
DECLARE
```

```
    v_emp_id employees.employee_id%TYPE := &cod_empleado;
```

```
    v_salary employees.salary%TYPE;
```

```
    v_divisor NUMBER;
```

```
BEGIN
```

```
    SELECT salary INTO v_salary FROM employees WHERE employee_id = v_emp_id;
```

```
    -- Empezamos desde la mitad del salario hacia abajo
```

```
    FOR i IN REVERSE 1 .. TRUNC(v_salary / 2) LOOP
```

```
        IF MOD(v_salary, i) = 0 THEN
```

```
            v_divisor := i;
```

```
            EXIT; -- El primero que encontremos de arriba a abajo es el mayor
```

```
        END IF;
```

```
    END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE('El salario es ' || v_salary || ' y su mayor divisor es ' || v_divisor);
```

```
END;
```

```
/
```

--7. Dado un país introducido por teclado, obtener el número de empleados que hay en ese país.

```
DECLARE
```

```
    v_pais countries.country_name%type := '&NOMBRE_PAIS';
```

```
    v_num_empleados NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(e.employee_id)
```

```
    INTO v_num_empleados
```

```
    FROM employees e
```

```
    JOIN departments d ON d.department_id = e.department_id
```

```
    JOIN locations l ON l.location_id = d.location_id
```

```
    JOIN countries c ON c.country_id = l.country_id
```

```
    WHERE LOWER(c.country_name) = LOWER(v_pais);
```

```
    DBMS_OUTPUT.PUT_LINE('El número de empleados de ' || v_pais || ' es: ' || v_num_empleados);
```

```
END;
```

```
/
```

--8. Crear una tabla llamada TANGULOS con tres columnas ángulo, seno, coseno. Rellenar la misma mediante un bloque PL de todos los ángulos comprendidos entre 0 y 90, en intervalos de diez en diez.

```
CREATE TABLE TANGULOS(
```

```
    angulo NUMBER(3) PRIMARY KEY,
```

```
    seno NUMBER(6,4),
```

```
    coseno NUMBER(6,4));
```

```
DECLARE
```

```
    v_angulo NUMBER := 0;
```

```
BEGIN
```

```
    LOOP
```

```
        EXIT WHEN v_angulo > 90;
```

```
        INSERT INTO TANGULOS(angulo, seno, coseno)
```

```
        VALUES (v_angulo, SIN(v_angulo), COS(v_angulo));
```

```
        v_angulo := v_angulo + 10;
```

```
    END LOOP;
```

```
    --COMMIT;
```

```
END;
```

```
/
```

--9. Haz un bloque anónimo que pida un id de empleado y cuente el número de vocales que hay en su email.

```
DECLARE
    v_emp_id employees.employee_id%TYPE := &id_empleado;
    v_email employees.email%TYPE;
    v_vocales NUMBER := 0;
BEGIN
    SELECT email INTO v_email FROM employees WHERE employee_id = v_emp_id;

    FOR i IN 1..LENGTH(v_email) LOOP
        IF UPPER(SUBSTR(v_email, i, 1)) IN ('A', 'E', 'I', 'O', 'U') THEN
            v_vocales := v_vocales + 1;
        END IF;
    END LOOP;
    -- REGEXP_COUNT busca el patrón (vocales) ignorando mayúsculas/minúsculas ('i')
    --v_vocales := REGEXP_COUNT(v_email, '[AEIOU]', 1, 'i');

    DBMS_OUTPUT.PUT_LINE('El email ' || v_email || ' tiene ' || v_vocales || ' vocales.');
```

END;

/

--10. Los departamentos de RRHH e ADMINISTRACIÓN se van a fusionar, por lo que debemos hacer un proceso que:

--10.a. Genere un nuevo departamento (RRHH+ADMIN)

--10.b. Cuyo responsable y localización será igual al del actual de RRHH

--10.c. Asigne a los trabajadores de ambos departamentos el nuevo manager y el nuevo

departamento

DECLARE

v_RRHH_manager departments.manager_id%type;

v_RRHH_location departments.location_id%type;

v_new_department_id departments.department_id%type := departments_seq.NEXTVAL;

BEGIN

-- Crear el nuevo dpto

--obtener manager y loc de RRHH

SELECT manager_id, location_id

INTO v_RRHH_manager, v_RRHH_location

FROM departments

WHERE

department_name = 'Human Resources';

--Insertar los datos del nuevo dpto

INSERT INTO departments (department_id, department_name, manager_id, location_id)

VALUES (v_new_department_id, 'RRHH+ADMIN', v_RRHH_manager, v_RRHH_location);

--Asignar a los trabajadores de ambos dptos el nuevo manager y depto

UPDATE employees

SET department_id = v_new_department_id,

manager_id = v_RRHH_manager

WHERE department_id IN (

SELECT department_id

FROM departments

WHERE department_name = 'Human Resources' OR department_name = 'Administration');

--COMMIT;

END;

/